M2103 PROGRAMMATION PAR OBJETS

TD 3. INTERFACES

JSHELL – PRINCIPE

- Java 9 et plus
- *Shell* (interprète de commande REPL) permettant de faire du Java interactif
- Comme python, ocaml, scala…
- Avantage : pas besoin de lancer Netbeans ou un IDE, de créer une hiérarchie de classes (ou le moindre fichier)
- Limité à des essais cours
- Commande *jshell* (par exemple sur *info-ssh1.iut.u-bordeaux.fr*)

JSHELL - EXEMPLE

```
$ jshell
|  Welcome to JShell -- Version 11.0.9.1
|  For an introduction type: /help intro

jshell> 2+2
$1 ==> 4

jshell> ArrayList<String> a = new ArrayList<>()
a ==> []
```

JSHELL - COMMANDES

- `/exit`
  `/help`
  `/list`
  `/k`
  `/save file`
  `/open file`
  `/edit`
  `/set editor command`

TRIER UN ARRAYLIST DE CHAÎNES

- Essayer :

```
jshell> var tableau = new ArrayList<String>();
jshell> tableau.add("Joe")
…
jshell> Collections.sort(tableau)
jshell> tableau
```

- Résultat trié,
- Par ordre lexicographique sur les chaînes

TRIER UN ARRAYLIST DE CHAÎNES – EXPLICATION

- sort est définie dans java.util.Collections
- Méthode de classe prenant le tableau en paramètre
- Action qui modifie le tableau
- Ordre dépendant du type des objets contenus
- String comprend donc une méthode de comparaison

TRIER UN ARRAYLIST D'OBJETS - 1

- Définir une classe maison :

```
jshell> class Personne {  String nom; int age ;
   ...>    Personne(String n, int a) {
   ...>      nom = n; age = a;
   ...>    }
   ...>    @Override
   ...>    public String toString() {
   ...>       return nom + " (" + age + ")";
   ...> }
   ...> }
```

- Placer des objets dans un ArrayList :

```
jshell> var people = new ArrayList<Personne>()
jshell> people.add(new Personne("Bill", 33))
```

TRIER UN ARRAYLIST D'OBJETS – 2

- Essai :

```
jshell> Collections.sort(people)
```

- Que se passe-t-il et pourquoi ?

TRIER UN ARRAYLIST D'OBJETS - 3

- Rendre des éléments comparables :
  - Déclarer qu'ils le sont :

```
class Personne implements Comparable<Personne> {
```

  - Faire en sorte qu'ils le soient vraiment :

```
    @Override
     int compareTo(Personne autre) {
         return this.age - autre.age;
   }
```

L'INTERFACE COMPARABLE<T>

- Voici cette interface

```
package java.lang;
public interface Comparable<T> {
      int compareTo(T other);
}
```

- Type générique : paramétré par T (comme ArrayList<T>…)
- Déclare que « ce qui se dit comparable doit disposer d'une méthode CompareTo »

QU'EST-CE QU'UNE INTERFACE ?

- Concept objet au sens de Java (il y a *beaucoup* d'autres sens à « interface »…)
- Code source contenant une **liste de signatures** de méthodes

```
interface Truc {
    int  chose();           // signature 1
    void machin();          // signature 2
}
```

- Liste les appels qu'il sera possible de faire sur ces objets
- Par défaut, les méthodes sont **publiques**

IMPLÉMENTER UNE INTERFACE

- Déclaration : `class Bidule implements Truc {`
- Définition :

```
class Bidule implements Truc {
    ...
    @Override
    public int chose() {
        return 33;
    }
    @Override
    public void machin() {
        System.out.println("Abra !");
    }
}
```

- Pas de limites au nombre d'interfaces implémentées : `class A implements B, C, D {`

ÉTENDRE UNE INTERFACE

- Par exemple :

```
interface A {
    void a();
}
interface B extends A {
    void b();
}
```

- (Les classes implémentant `B` doivent donner du code pour `b()` et `a()`)

UTILISER UNE INTERFACE

- Une interface est un type
  ```
  Truc v;
  ```
- Pour un objet de ce type, il faut une classe qui implémente l'interface
  ```
  v = new Machin();
  ```
- On peut ensuite appeler les méthodes décrites dans l'interface
  ```
  v.machin();
  ```

POLYMORPHISME

- Nous en reparlerons…

```
interface Animal {
   void saluer(String nom);
}
class Chat implements Animal {
   @Override
   void saluer(String nom) {
      System.out.println("Miaou " + nom + " ! ");
   }
}
class Poisson implements Animal {
   @Override
   void saluer(String nom) {
      System.out.println("Bloup.");
   }
}
Animal[] animaux = { new Poisson(), new Chat(), new Poisson() };
for (Animal a : animaux) {
   a.saluer("Patron");
}
```